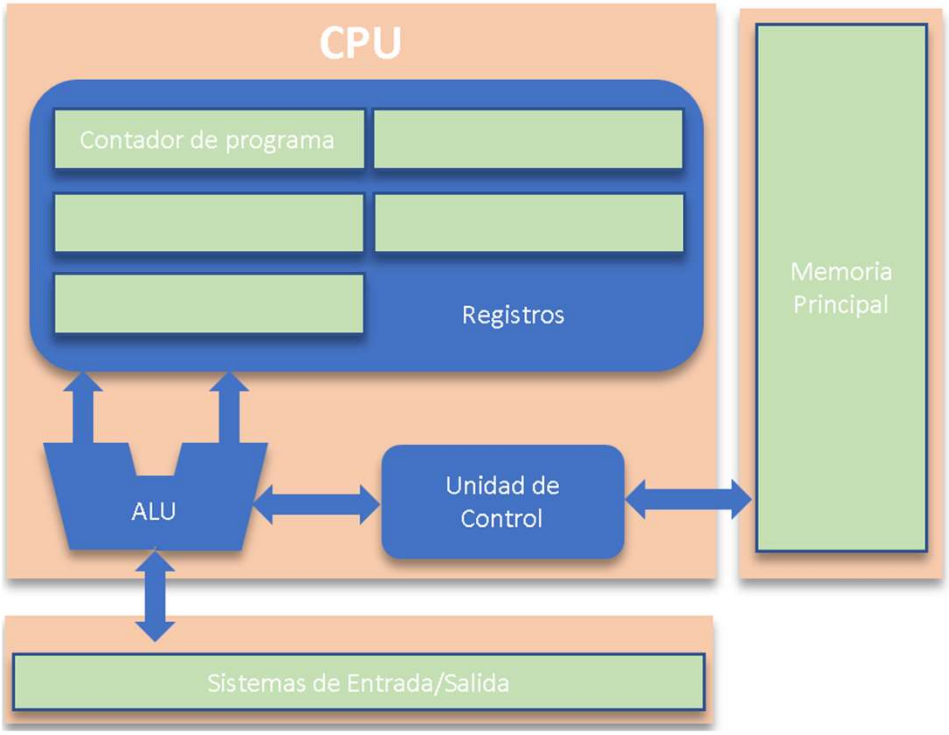
MIPS Microprocessor without Interlocked Pipeline Stages

12026

Juan Camilo Acevedo Bedoya | Facultad de Ingenierías

Arquitectura de Computadores

La arquitectura del computador se refiere al diseño conceptual y la estructura operativa fundamental de un sistema de cómputo. Esto incluye el conjunto de instrucciones (ISA), el diseño del hardware y los componentes del sistema, así como las técnicas utilizadas para gestionar y procesar datos. La arquitectura del computador define cómo los diferentes componentes de hardware y software interactúan para ejecutar programas eficientemente.



Componentes Clave de la Arquitectura del Computador

1. **Unidad Central de Procesamiento (CPU):** El cerebro del computador, responsable de ejecutar instrucciones.

2. **Memoria:** Donde se almacenan los datos y programas que está utilizando el computador.

3. **Unidades de Entrada/Salida (I/O):** Permiten la interacción con el entorno exterior (teclados, ratones, impresoras, etc.).

4. **Buses:** Líneas de comunicación que transportan datos entre diferentes componentes del computador.

Ejemplos de Arquitectura

1. Arquitectura Von Neumann:

- **Descripción:** Define un sistema de computación en el cual la CPU, la memoria y los dispositivos de entrada/salida están conectados por buses. Utiliza el mismo espacio de memoria para instrucciones y datos.
- **Ejemplo:** Computadoras personales tradicionales.

2. Arquitectura Harvard:

- **Descripción:** Separación de la memoria de instrucciones y la memoria de datos, permitiendo accesos simultáneos y mejorando la velocidad.
- **Ejemplo:** Microcontroladores y DSP (Digital Signal Processors).

Ejemplos de Arquitectura

3. Arquitectura RISC (Reduced Instruction Set Computer):

- **Descripción:** Utiliza un conjunto de instrucciones reducido y optimizado para aumentar la velocidad y eficiencia. Los procesadores RISC generalmente ejecutan una instrucción por ciclo de reloj.
- **Ejemplo:** ARM, MIPS, SPARC.

4. Arquitectura CISC (Complex Instruction Set Computer):

- **Descripción:** Tiene un conjunto de instrucciones más amplio y complejo, lo que puede reducir el número de instrucciones necesarias para realizar tareas complejas.
- **Ejemplo:** Intel x86, VAX

Ejemplos de Arquitectura

5. Arquitectura EPIC (Explicitly Parallel Instruction Computing):

- **Descripción:** Permite ejecutar múltiples instrucciones en paralelo mediante la especificación explícita de la paralelización en el código.
- **Ejemplo:** Intel Itanium.

6. Arquitectura VLIW (Very Long Instruction Word):

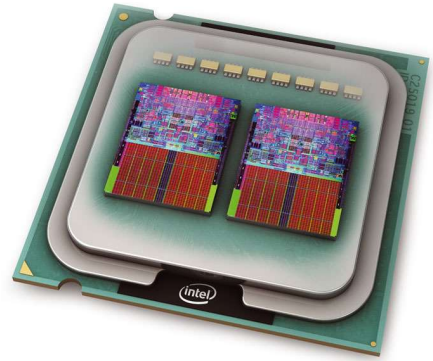
- **Descripción:** Procesadores que pueden ejecutar múltiples operaciones en paralelo dentro de una sola instrucción larga.
- **Ejemplo:** Transmeta Crusoe, algunos DSP.

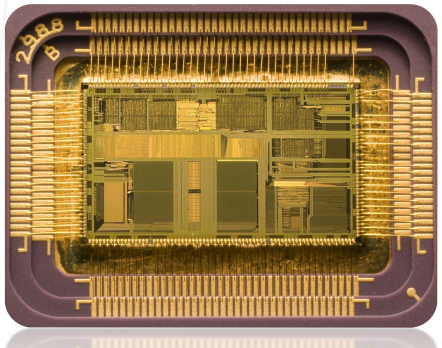
Aplicaciones de Diferentes Arquitecturas

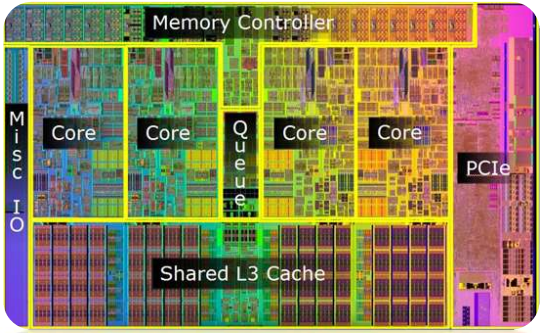
- **Computadoras Personales y Servidores:** Generalmente utilizan arquitecturas CISC (como Intel x86) debido a su flexibilidad y compatibilidad con una amplia gama de aplicaciones.
- **Dispositivos Móviles y Embebidos:** Suelen utilizar arquitecturas RISC (como ARM) por su eficiencia energética y rendimiento.
- **Supercomputadoras y Sistemas de Alto Rendimiento**: Pueden utilizar arquitecturas especializadas como EPIC o VLIW para maximizar el paralelismo y el rendimiento.

¿Qué es un Procesador?

Un procesador, también conocido como Unidad Central de Procesamiento (CPU, por sus siglas en inglés), es el componente principal de un computador que interpreta y ejecuta instrucciones de programas. Es el cerebro del computador, encargado de realizar cálculos, tomar decisiones y gestionar la transferencia de datos entre los diferentes componentes del sistema.







Funciones Principales del Procesador

1. Ejecución de Instrucciones:

- Lee instrucciones desde la memoria.
- Decodifica las instrucciones para determinar las operaciones a realizar.
- Ejecuta las operaciones utilizando las unidades aritmético-lógicas (ALU).

2. Control de Flujo de Datos:

- Gestiona el flujo de datos entre la memoria y los periféricos.
- Utiliza registros internos para almacenar temporalmente datos y resultados intermedios.

3. Toma de Decisiones:

- Realiza operaciones de comparación y toma decisiones basadas en los resultados.
- Ejecuta instrucciones de salto condicional y bucles en los programas.

Componentes Clave de un Procesador

1. Unidad de Control (CU):

- Interpreta las instrucciones de los programas y genera señales de control para coordinar las operaciones de los otros componentes del procesador.

2. Unidad Aritmético-Lógica (ALU):

- Realiza operaciones aritméticas (suma, resta, multiplicación, división) y lógicas (AND, OR, NOT, XOR).

3. Registros:

- Pequeñas áreas de almacenamiento rápido dentro del procesador que se utilizan para almacenar datos temporales y direcciones

Componentes Clave de un Procesador

4. Caché:

- Memoria de alta velocidad ubicada dentro del procesador que almacena temporalmente datos e instrucciones de uso frecuente para acelerar el acceso.

5. Buses:

- Conjunto de líneas de comunicación que transportan datos, direcciones y señales de control entre el procesador y otros componentes del sistema.

Tipos de Procesadores

1. Procesadores de Uso General:

- **CPU de Computadoras Personales:** Intel Core, AMD Ryzen.
- **Procesadores de Servidores:** Intel Xeon, AMD EPYC.

2. Procesadores Especializados:

- **Microcontroladores:** Utilizados en sistemas embebidos, como el Arduino (ATmega328).
- **Procesadores de Señales Digitales (DSP):** Especializados en el procesamiento de señales, como el Texas Instruments TMS320.
- **Unidades de Procesamiento Gráfico (GPU):** Optimizados para operaciones en paralelo, como NVIDIA GeForce, AMD Radeon.

Tipos de Procesadores

3. Procesadores Basados en RISC:

- **ARM:** Utilizados en dispositivos móviles y sistemas embebidos.
- **MIPS:** Utilizados en sistemas embebidos y aplicaciones industriales.

4. Procesadores Basados en CISC:

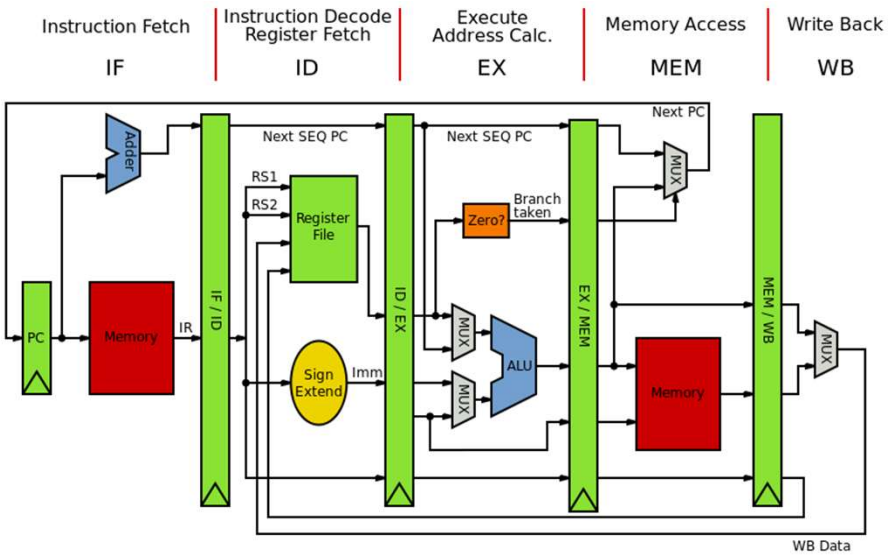
- **Intel x86:** Dominan el mercado de computadoras personales y servidores.

MIPS Microprocessor without Interlocked Pipeline Stages

MIPS (Microprocessor without Interlocked Pipeline Stages) es una arquitectura de conjunto de instrucciones (ISA) desarrollada por MIPS Computer Systems en 1980 (actualmente MIPS Technologies). MIPS es conocida por su simplicidad y eficiencia, y ha sido ampliamente utilizada en sistemas embebidos y computadoras.

Las primeras arquitecturas MIPS eran de 32 bits, con versiones de 64 bits añadidos más tarde.

Existen múltiples revisiones del conjunto de instrucciones MIPS, incluyendo MIPS I, MIPS II, MIPS III, MIPS IV, MIPS V, MIPS32 y MIPS64 - Las revisiones actuales son MIPS32 y MIPS64.

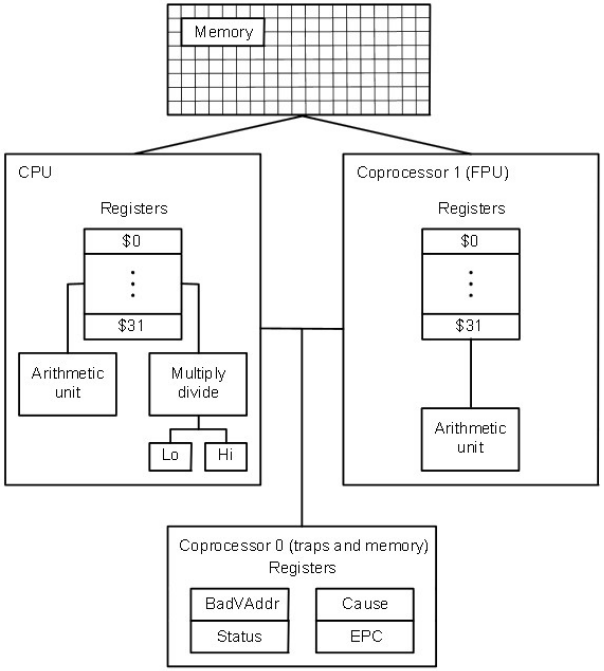


Características clave de la arquitectura MIPS

- **RISC (Reduced Instruction Set Computer):** MIPS es una arquitectura RISC, lo que significa que utiliza un conjunto reducido de instrucciones altamente optimizadas.
- **Instrucciones de longitud fija:** Todas las instrucciones MIPS tienen una longitud fija de 32 bits.
- **Registros de propósito general:** MIPS tiene 32 registros de propósito general (R0-R31).
- **Pipeline eficiente:** La arquitectura MIPS se diseñó para un procesamiento de instrucciones eficiente mediante un pipeline de cinco etapas.

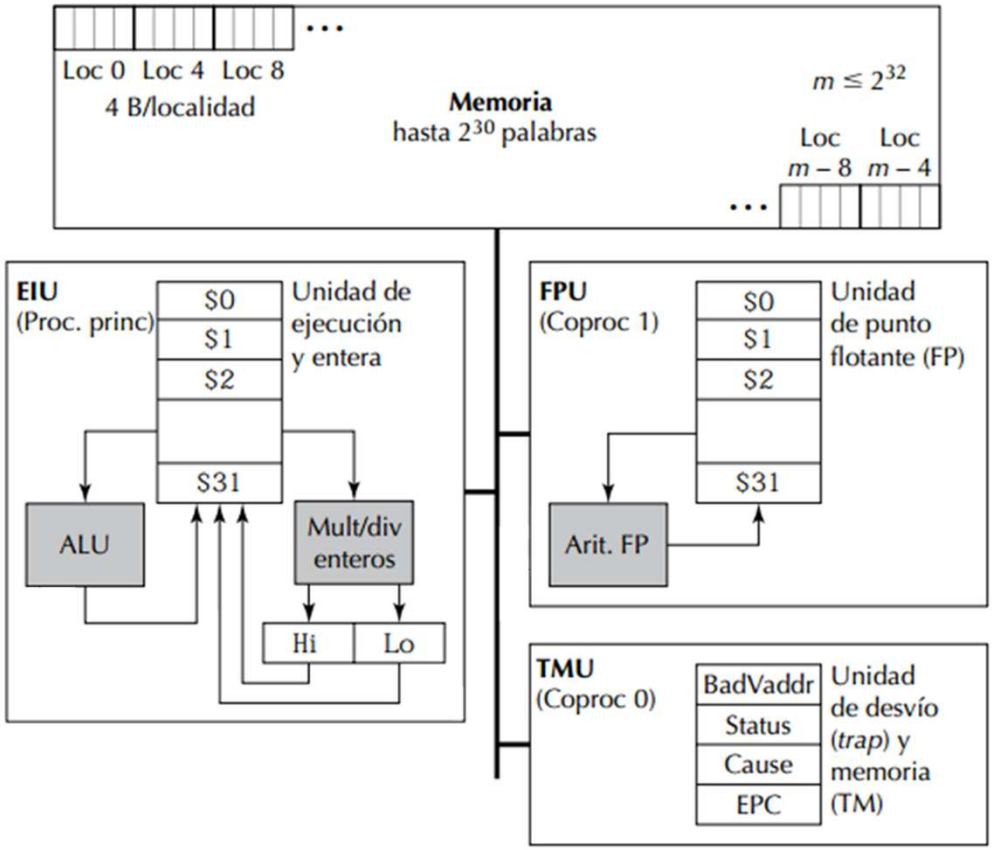
Componentes de la arquitectura MIPS

- **Unidad de Control (CU):** Coordina las operaciones de la CPU y controla la secuencia de instrucciones.
- **Unidad Aritmético-Lógica (ALU):** Realiza operaciones matemáticas y lógicas.
- **Memoria de Datos:** Almacena datos y permite la lectura y escritura.
- **Memoria de Instrucciones:** Contiene las instrucciones del programa.
- **Registros:** Los 32 registros de propósito general almacenan datos temporales.
- **Pipeline de cinco etapas:** Divide la ejecución de instrucciones en cinco etapas: búsqueda, decodificación, ejecución, acceso a memoria y escritura



A la Verdad, por la fe y la ciencia

Componentes de la arquitectura MIPS

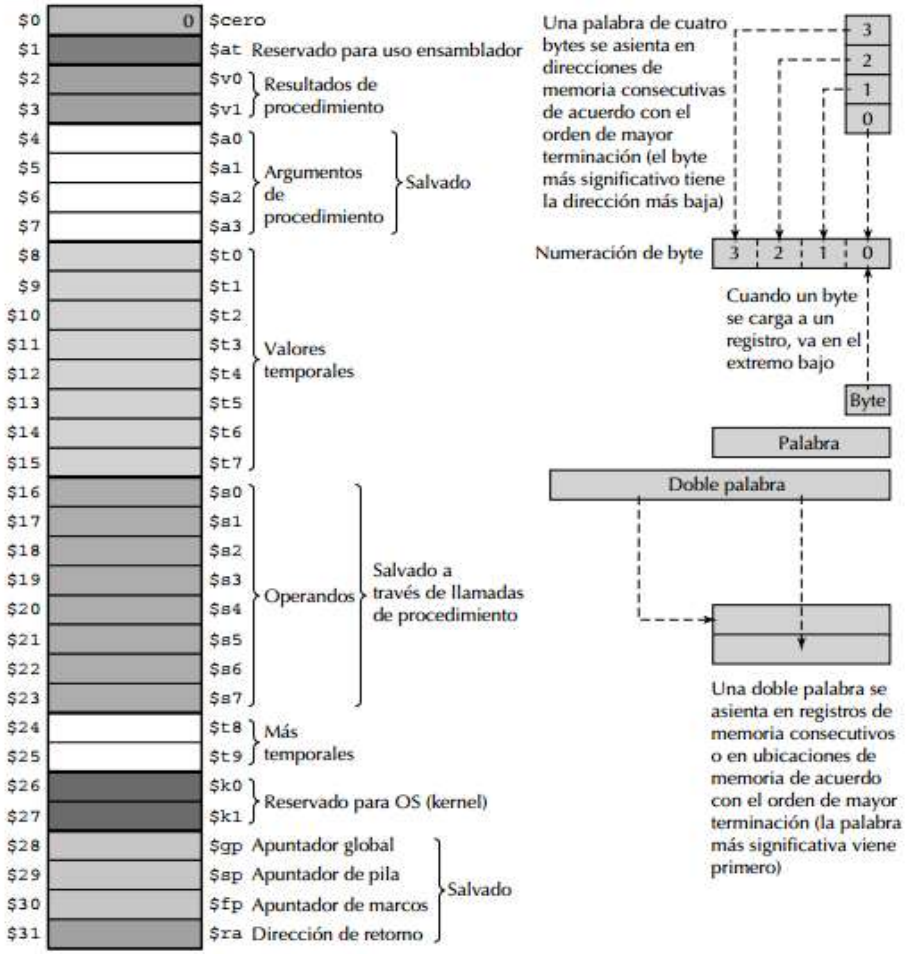


Registros

Los registros en la arquitectura MIPS son elementos de almacenamiento rápidos y temporales dentro del procesador que se utilizan para realizar operaciones de cálculo y manipulación de datos de manera eficiente. MIPS utiliza 32 registros de propósito general, cada uno con una capacidad de 32 bits. Estos registros se nombran desde $0 hasta $31 y se utilizan para diferentes propósitos específicos dentro de la ejecución de instrucciones.

- 32 registros de propósito general
- Registros propósito específico:
- HI-LO: para productos y divisiones
- PC.
- Ancho de los registros: 32 bits

Registros


$0 0 $cero
$1 $at Reservado para uso ensamblador
$2 $v0
$3 $v1
Resultados de procedimiento
$4 $a0
$5 $a1
$6 $a2
$7 $a3
Argumentos de procedimiento
Salvado
$8 $t0
$9 $t1
$10 $t2
$11 $t3
$12 $t4
$13 $t5
$14 $t6
$15 $t7
Valores temporales
$16 $s0
$17 $s1
$18 $s2
$19 $s3
$20 $s4
$21 $s5
$22 $s6
$23 $s7
Operandos
Salvado a través de llamadas de procedimiento
$24 $t8
$25 $t9
Más temporales
$26 $k0
$27 $k1
Reservado para OS (kernel)
$28 $gp Apuntador global
$29 $sp Apuntador de pila
$30 $fp Apuntador de marcos
$31 $ra Dirección de retorno
Salvado
Una palabra de cuatro bytes se asienta en direcciones de memoria consecutivas de acuerdo con el orden de mayor terminación (el byte más significativo tiene la dirección más baja)
3
2
1
0
Numeración de byte
3 2 1 0
Cuando un byte se carga a un registro, va en el extremo bajo
Byte
Palabra
Doble palabra
Una doble palabra se asienta en registros de memoria consecutivos o en ubicaciones de memoria de acuerdo con el orden de mayor terminación (la palabra más significativa viene primero)

Registros

En MIPS los registros $0 a $31 son de propósito general pueden emplearse para contener datos o punteros. Existe un convenio que dota de pseudónimos y usos determinados a todos los registros del MIPS:

$s0 ... $s7 ($16 ... $23): respetados en llamadas a subprogramas.
$t0 ... $t9 ($8 ... $15, $24 y $25): no respetados en llamadas a subprogramas.
$v0 y $v1 ($2 y $3): valores de retorno de subprogramas.
$a0 ... $a3 ($4 ... $7): argumentos para subprogramas.
$zero ($0): valor constante 0 (no modificable).
$gp ($28): puntero a zona de datos globales.
$sp ($29): puntero de pila.

Registros

En MIPS los registros $0 a $31 son de propósito general pueden emplearse para contener datos o punteros. Existe un convenio que dota de pseudónimos y usos determinados a todos los registros del MIPS:

$fp ($30): puntero a bloque de activación.
$ra ($31): dirección de retorno (usado implícitamente en la instrucción jal).
$at ($1): uso interno del traductor de ensamblador en pseudoinstrucciones.
$k0 y $k1 ($26 y $27): uso privado del núcleo del sistema operativo (kernel).

Los registros $s0 ... $s7, $t0 ... $t9, $a0 ... $a3, $v0 y $v1
Pueden contener tanto datos de cualquier tipo como punteros.

Registros

Nombre	Registro Número	Uso
$zero	0	Valor constante cero (0)
$at	1	Reservado para uso del ensamblador
$v0 - $v1	2 - 3	Valores para resultados y evaluación de expresiones
$a0 - $a3	4 - 7	Argumentos
$t0 - $t7	8 - 15	Temporales
$s0 - $s7	16 - 23	Salvados o guardados
$t8 - $t9	24 - 25	Más temporales
$gp	28	Puntero global
$sp	29	Puntero de pila (Stack Pointer)
$fp	30	Frame Pointer
$ra	31	Dirección de retorno

De código a Bits

LENGUAJE DE ALTO NIVEL

```
swap (int v [], int k)
{
    temp = v[k];
    v[k] = v[k+1];
    v[k+1] = temp;
}
```

Compilador

LENGUAJE ENSAMBLADOR

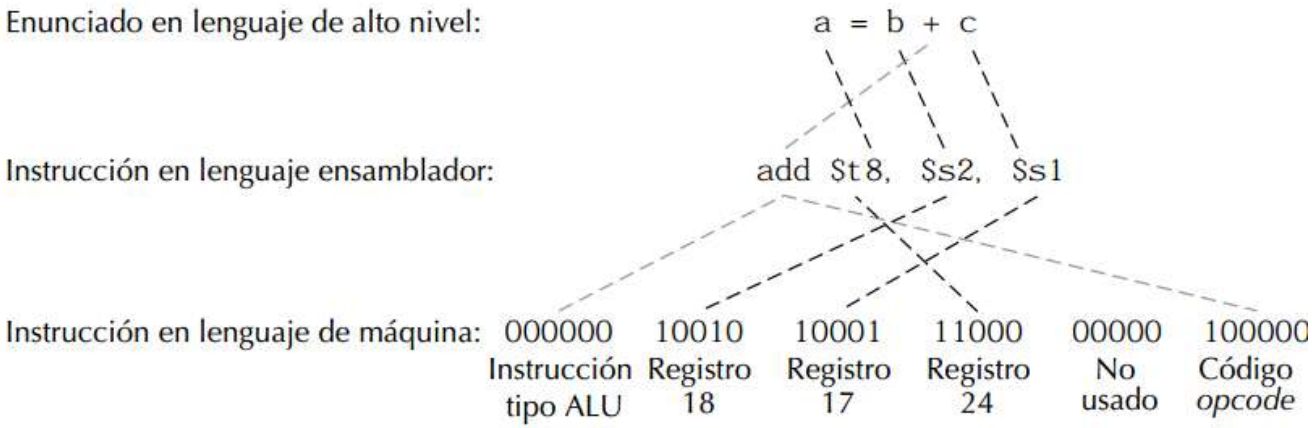
```
swap:
        muli  $2, $5, 4
        add   $2, $4, $2
        lw    $15, 0($2)
        lw    $16, 4($2)
        sw    $16, 0($2)
        sw    $15, 4($2)
        jr    $31
```

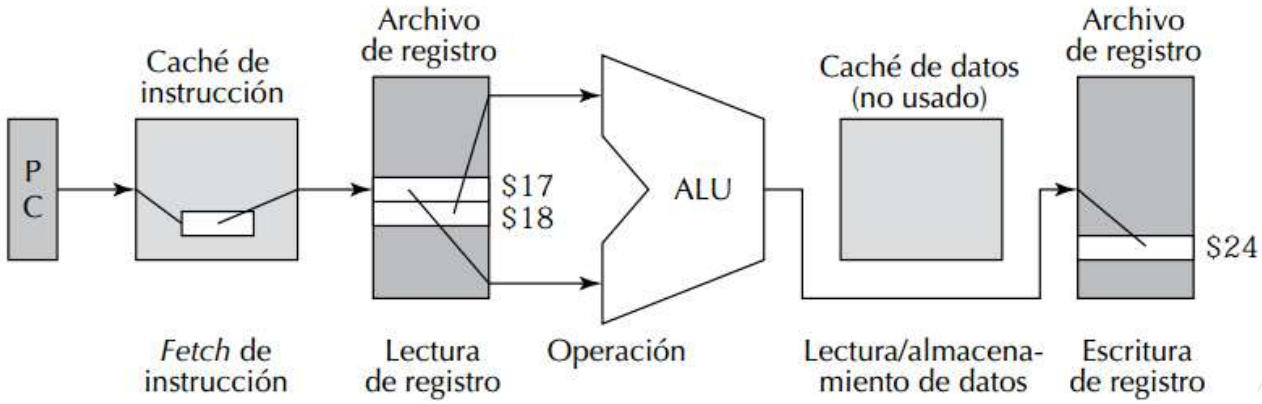
Ensamblador

LENGUAJE DE MAQUINA

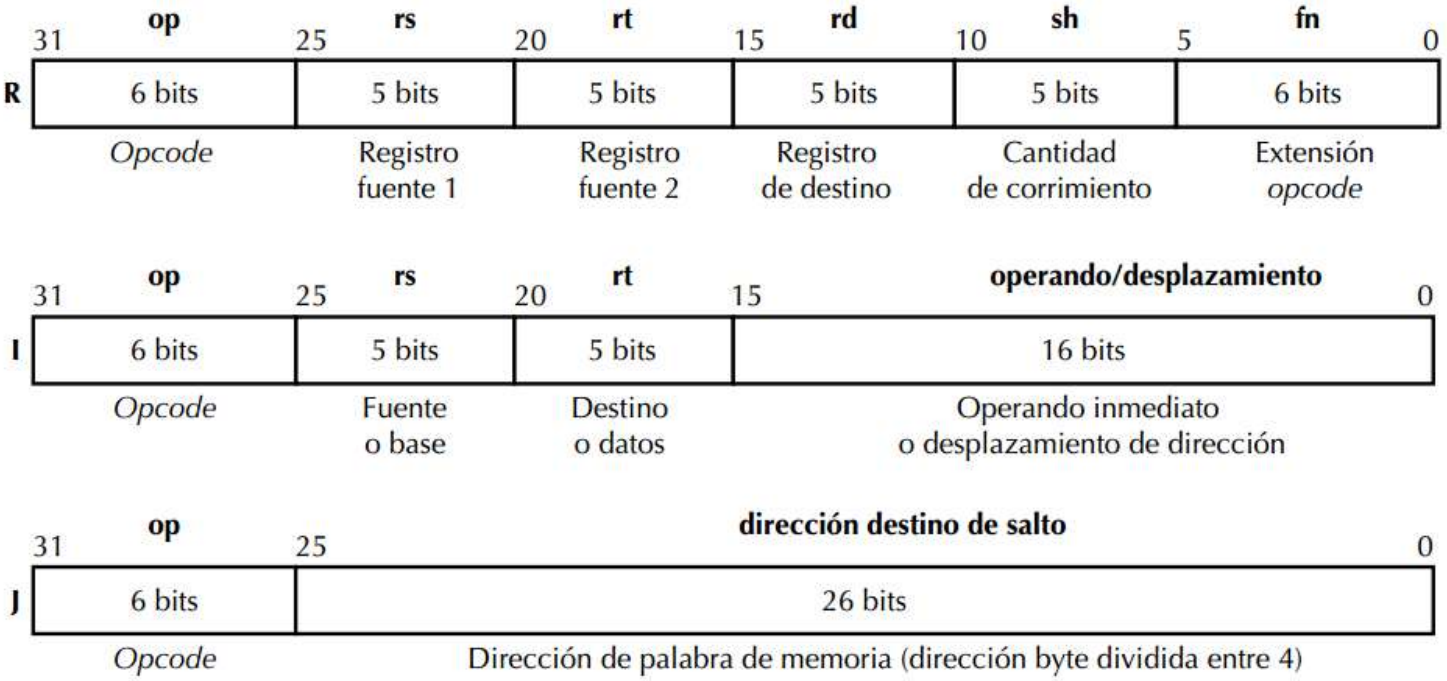
```
00000001010000000100000011000
00000001000111000011000010001
10001100011000100000000000000
10001100111100100000000000100
10101100111100100000000000000
10101100011000100000000000100
00000011111000000000000001000
```

Formato de Instrucción





Formato de Instrucción



A la Verdad, por la fe y la ciencia

Instrucciones MIPS

Clase	Instrucción	Uso		Significado	op	fn
Copia	Load upper immediate	lui	rt,imm	rt ← (imm, 0X0000)	15	
Aritmética	Add	add	rd,rs,rt	rd ← (rs) + (rt)	0	32
	Subtract	sub	rd,rs,rt	rd ← (rs) − (rt)	0	34
	Set less than	slt	rd,rs,rt	rd ← if (rs) < (rt) then 1 else 0	0	42
	Add immediate	addi	rt,rs,imm	rt ← (rs) + imm	8	
	Set less than immediate	slti	rd,rs,imm	rd ← if (rs) < imm then 1 else 0	10	
Lógica	AND	and	rd,rs,rt	rd ← (rs) ∧ (rt)	0	36
	OR	or	rd,rs,rt	rd ← (rs) ∨ (rt)	0	37
	XOR	xor	rd,rs,rt	rd ← (rs) ⊕ (rt)	0	38
	NOR	nor	rd,rs,rt	rd ← ((rs) ∨ (rt))′	0	39
	AND immediate	andi	rt,rs,imm	rt ← (rs) ∧ imm	12	
	OR immediate	ori	rt,rs,imm	rt ← (rs) ∨ imm	13	
	XOR immediate	xori	rt,rs,imm	rt ← (rs) ⊕ imm	14	
Acceso de memoria	Load word	lw	rt,imm(rs)	rt ← mem[(rs) + imm]	35	
	Store word	sw	rt,imm(rs)	mem[(rs) + imm] ← (rt)	43	
Transferencia de control	Jump	j	L	goto L	2	
	Jump register	jr	rs	goto (rs)	0	8
	Branch less than 0	bltz	rs,L	if (rs) < 0 then goto L	1	
	Branch equal	beq	rs,rt,L	if (rs) = (rt) then goto L	4	
	Branch not equal	bne	rs,rt,L	if (rs) ≠ (rt) then goto L	5	

Seudoinstrucciones MIPS

Clase	Seudoinstrucción	Uso		Significado
Copia	Move	`move`	`regd,regs`	regd ← (regs)
	Load address	`la`	`regd,address`	carga dirección calculada, no contenido
	Load immediate	`li`	`regd,anyimm`	regd ← valor inmediato arbitrario
Aritmética	Absolute value	`abs`	`regd,regs`	regd ← |(regs)|
	Negate	`neg`	`regd,regs`	regd ← −(regs); con desbordamiento
	Multiply (into register)	`mul`	`regd,reg1,reg2`	regd ← (reg1) × (reg2); sin desbordamiento
	Divide (into register)	`div`	`regd,reg1,reg2`	regd ← (reg1) ÷ (reg2); con desbordamiento
	Remainder	`rem`	`regd,reg1,reg2`	regd ← (reg1)mod(reg2)
	Set greater than	`sgt`	`regd,reg1,reg2`	regd ← si (reg1) > (reg2) entonces 1 de otro modo 0
	Set less or equal	`sle`	`regd,reg1,reg2`	regd ← si (reg1) ≤ (reg2) entonces 1 de otro modo 0
	Set greater or equal	`sge`	`regd,reg1,reg2`	regd ← si (reg1) ≥ (reg2) entonces 1 de otro modo 0
Corrimiento	Rotate left	`rol`	`regd,reg1,reg2`	regd ← (reg1) rotado izquierdo por (reg2)
	Rotate right	`ror`	`regd,reg1,reg2`	regd ← (reg1) rotado derecho por (reg2)
Lógica	NOT	`not`	`reg`	reg ← (reg)′
Acceso a memoria	Load doubleword	`ld`	`regd,address`	carga *regd* y el registro siguiente
	Store doubleword	`sd`	`regd,address`	almacena *regd* y el registro siguiente
Transferencia de control	Branch less than	`blt`	`reg1,reg2,L`	si (reg1) < (reg2) entonces ir a L
	Branch greater than	`bgt`	`reg1,reg2,L`	si (reg1) > (reg2) entonces ir a L
	Branch less or equal	`ble`	`reg1,reg2,L`	si (reg1) ≤ (reg2) entonces ir a L
	Branch greater or equal	`bge`	`reg1,reg2,L`	si (reg1) ≥ (reg2) entonces ir a L

Directivas de Ensamblador

```
            .macro                  # Iniciar macro
            .end_macro              # Fin de macro
            .text                   # Inicia segmento de texto de programa
            .data                   # Inicia segmento de datos del programa
tiny:       .byte    156, 0x7a      # Dato tipo byte
max:        .word    35000          # Dato tipo entero
small:      .float   2E-3           # Dato tipo flotante sencillo
big:        .double  2E-3           # Dato tipo flotante doble
            .align   2              # Alinea el siguiente objeto en frontera de palabra

array:      .space   600            # Reserva 600 bytes = 150 palabras
str1:       .ascii   “a*b”          # Cadena de texto
str1:       .asciiz  “xyz”          # Cadena de texto con terminación nula
            .global  main           # Considera main un nombre global
```

Funciones de Entrada y Salida. Syscall

Clase	($v0)	Función	Argumento(s)	Resultado
Salida	1	Imprimir entero	Entero en $a0	Muestra un entero
	2	Imprimir punto flotante sencillo	Flotante sencillo en $f12	Muestra un flotante sencillo
	3	Imprimir punto flotante doble	Flotante doble en $f12, $f13	Muestra un flotante doble
	4	Imprimir cadena	Apuntador en $a0	Muestra una cadena termina en nulo
Entrada	5	Leer entero		Regresa entero en $v0
	6	Leer punto flotante sencillo		Regresa flotante sencillo en $f0
	7	Leer punto flotante doble		Regresa flotante doble en $f0, $f1
	8	Leer cadena	Apuntador en $a0, longitud en $a1	Regresa cadena en el buffer del apuntador

UCO
Universidad Católica de Oriente
Vigilada Mineducación

Renovación Acreditación Institucional en ALTA CALIDAD

¡Gracias!